DALHOUSIE UNIVERSITY

Predictive Analytics

Targets, predictors, and classification metrics

Prof. Gabriel Spadon, PhD

Faculty of Computer Science

spadon@dal.ca

1 Predictive analytics

What it is for Predictive analytics uses the variables you can measure to estimate the one you cannot. It belongs to the *Data Modelling* stage of CRISP-DM[1], where predictors are used to estimate a clearly defined target variable, and so there is something to be measurably wrong about.

1.1 Motivation

In many applications it is useful to predict one variable using another, or a set of other variables. Three examples follow.

- Diagnose a patient based on their blood sample results.
- Predict the ranking of a video game player based on their location and the hours they played.
- Detect a fraudulent transaction based on the time, amount transferred and location.

In each example the inputs are cheap, plentiful and already recorded. The output is expensive, delayed, or unknown at the moment the decision has to be made. Figure 1 shows this for the diagnosis example.

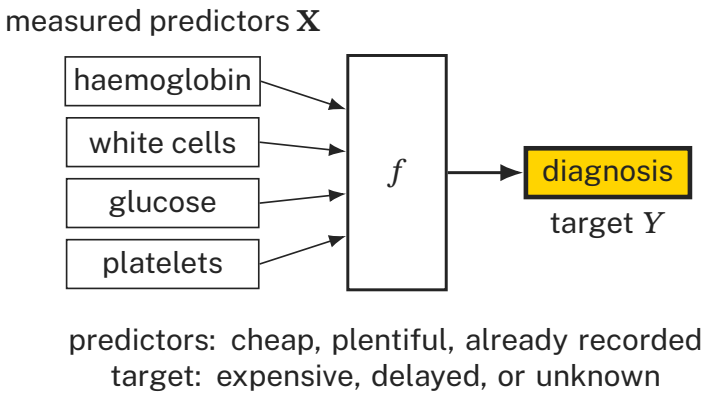


Figure 1. Measured predictors feed an unknown function f that produces the target, here a diagnosis.

1.2 Target and predictors

The variable Y we want to predict is called the **target** variable, the **response** variable, the **outcome**, or sometimes the **dependent** variable. In classification it is often called the **label** variable. It may be more important, more difficult to obtain, and/or influenced directly or indirectly by the other variables we measured.

The variables $\mathbf{X} = X_1, X_2, \ldots, X_p$ we use to predict Y are called the **predictors**, descriptors, explanatory variables, features, attributes, or covariates. They may be easier to obtain, and we suspect they describe the observations in a way that somehow "explains" the target variable.

The names differ by discipline, not by meaning. Statistics says *response* and *covariates*. Machine learning says *label* and *features*. This course says *target* and *predictors*.

1.3 Predictive modelling

We assume there is a relationship

$$Y = f(X_1, X_2, \ldots, X_p).$$

- **Given:** examples of observations where we measured Y and $\mathbf{X}$.
- **Goal:** find a model that approximates the unknown function f. A prediction model is any algorithm that estimates f.

Since we use examples of the mapping of $\mathbf{X}$ to Y to "learn" the function f, training a model to accomplish this task is also called **supervised learning**. The supervision is the y_i in the training examples. The training data carries the answers, and the algorithm is graded against them (Figure 2).

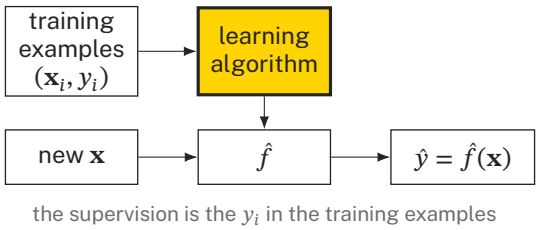


Figure 2. Supervised learning. Training examples with known y_i produce $\hat{f}$, which is then applied to a new $\mathbf{x}$.

f versus $\hat{f}$

f is the true, unknown relationship. $\hat{f}$ is what the algorithm produced from a finite sample. Every metric in this class measures $\hat{f}$ on data $\hat{f}$ has not seen. That score stands in for what we actually want to know, which is how close $\hat{f}$ is to f on data nobody has seen yet.

1.4 Why learn a model?

Predictive models have two main uses.

1. **Prediction.** Make predictions of outcome given the descriptors for new observations. It may not be as important that the functional form of the model is easily understandable by humans.
2. **Inference.** Use the models to better understand the factors that influence the outcome. Here obtaining the approximation of f is the most important outcome.

The two uses are scored differently. A prediction goal is scored on its error on cases not used to build the model. An inference goal is scored on whether the fitted form is stable and readable. A model can do well on one and badly on the other.

1.5 Types of modelling tasks

The type of modelling task depends on the type of the target variable Y.

- **Classification** task if Y is categorical (nominal or ordinal), for example, predict the species of a flower given measurements of its parts.
- **Regression** task if Y is numeric, for example, predict the market value of a house given its characteristics.

From section 3 onwards this class assumes a categorical target. Regression metrics are covered later.

2 Models and model selection

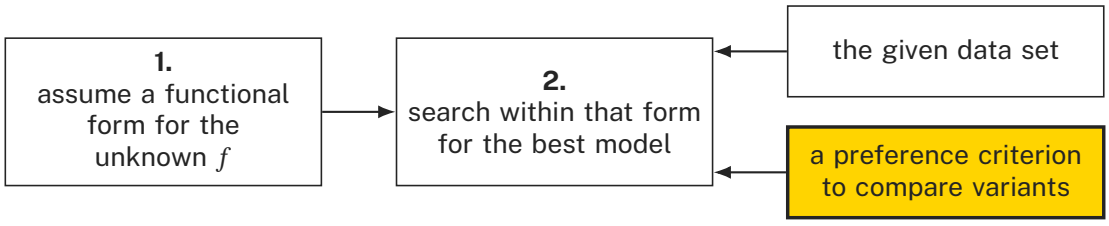
What it is for Many algorithms can estimate f. This section says what they have in common, how they differ, and how to choose between them.

2.1 Model assumptions

There are many algorithms that can be used to approximate f based on a data set. All have some key characteristics.

1. They assume a certain functional form for the unknown function f.
2. Given this assumed form, the algorithm will try to obtain the best possible model based on
   (a) the given data set, and
   (b) a certain preference criterion that allows the algorithm to compare the different alternative model variants and choose the best one.

The evaluation metric enters in step 2(b), *before* any model has been fitted (Figure 3). The metric is part of the search, not a report written afterwards.



The preference criterion is where the evaluation metric enters, before fitting any model.

Figure 3. Every algorithm assumes a form for f, then searches within it using the data and a preference criterion.

2.2 Different types of models

Examples include

- mathematical formulae, e.g. linear discriminants;
- logical approaches, e.g. classification or regression trees, rules;
- distance-based approaches, e.g. k-nearest neighbours;
- other approaches, e.g. support vector machines;
- sets of models (ensembles), e.g. random forests, adaBoost.

For classification evaluation, each classifier produces a predicted class for each test case. That is why one set of metrics serves all of them.

2.3 Pros and cons

Different models will have different

- assumptions on the unknown form of function f, which leads to different levels of flexibility in how they can approximate (or not) different types of functions;

- computational complexity of the search problem;
- amounts of data needed to achieve reasonable performance;
- interpretability of the resulting model.

No model family wins on all four. A flexible ensemble gains accuracy on complex functions at the price of data, compute and interpretability. A linear model is readable and stable on small samples, at the price of a restrictive assumption about the shape of f (Table 1).

Table 1. Qualitative trade-offs between a linear model and a flexible ensemble. Schematic comparison, not measured scores. The task and implementation matter.

	Linear model	Flexible ensemble
flexibility	lower	higher
data needed	lower	higher
compute	lower	higher
interpretability	higher	lower

2.4 Which model should I use?

This question is often known as the **model selection** problem, and the answer depends on the goals of the final user. It has three steps (Figure 4).

1. Establish the preference criteria for your problem. *Selecting an evaluation metric is part of this.*
2. Compare different models or variants of the same model.
3. Select the best model according to the preference criteria.

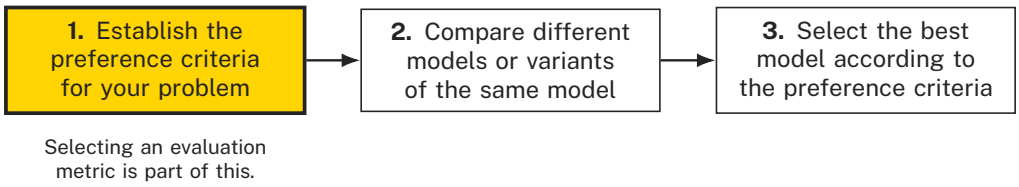


Figure 4. The three steps of model selection. The metric is fixed in step 1.

Step 1 comes first. If the metric is chosen after the results are in, steps 2 and 3 prove nothing, because with enough metrics every model is best on one of them. Write the criterion down in advance, in the report.

3 The running test set

What it is for Every metric up to the ROC curve (section 11) reads one table of counts. A single ten-case example is used throughout so that every number can be checked against the others.

A three-class model is evaluated on the ten test cases in Table 2.

Four of ten predictions are correct. The true class counts are c1: 4, c2: 3, c3: 3. The predicted class counts are c1: 5, c2: 2, c3: 3. The true counts are fixed by the data, and the predicted counts are chosen by the model. Almost every metric below is a ratio involving one or the other.

Table 2. True and predicted classes for the ten-case running example

case	1	2	3	4	5	6	7	8	9	10
true y_i	c1	c1	c2	c1	c3	c1	c2	c3	c2	c3
predicted $\hat{y}_i$	c1	c2	c1	c3	c3	c1	c1	c3	c1	c2
correct?	yes	no	no	no	yes	yes	no	yes	no	no

4 The error rate

What it is for The error rate is the share of test cases the model gets wrong. It is the simplest classification metric.

The formula Given a test set for which we know the true class, the error rate is

$$L_{0/1} = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} I(\hat{y}_i \neq y_i)$$

where $\hat{y}_i$ is the model's prediction for test case i, y_i is its true class, and I is 1 when the prediction is wrong and 0 otherwise. Accuracy is its complement, $1 - L_{0/1}$.

Worked example On the running example the indicator returns 1 on cases 2, 3, 4, 7, 9 and 10, so

$$L_{0/1} = \frac{1}{10}(0+1+1+1+0+0+1+0+1+1) = \frac{6}{10} = 0.6, \qquad \text{Accuracy} = 1 - L_{0/1} = 0.4.$$

What it cannot see $L_{0/1}$ is called the *zero-one loss* because each case contributes exactly 0 or 1. A near miss and a wild miss cost the same, and mistaking a benign case for a malignant one costs the same as the reverse. Section 6 removes that assumption.

5 Confusion matrices

What it is for The error rate is one number. The confusion matrix shows which classes are confused with which.

The definition A confusion matrix is a square $n_c \times n_c$ matrix, where n_c is the number of class values of the problem. The matrix contains the number of times each pair (*ObservedClass*, *PredictedClass*) occurred when testing a classification model on a set of cases, as in Table 3.

Table 3. General form of a three-class confusion matrix

		Predicted		
		c_1	c_2	c_3
Observed	c_1	$n_{1,1}$	$n_{1,2}$	$n_{1,3}$
	c_2	$n_{2,1}$	$n_{2,2}$	$n_{2,3}$
	c_3	$n_{3,1}$	$n_{3,2}$	$n_{3,3}$

Rows are the truth. Columns are the model. The diagonal $n_{j,j}$ holds the correct predictions. Every cell off the diagonal is an error, and its row tells you what was really there. The error rate is

$$L_{0/1} = 1 - \frac{\sum_j n_{j,j}}{\sum_j \sum_k n_{j,k}}.$$

Worked example The running example gives Table 4.

Table 4. Confusion matrix for the running example

		Predicted			
		c1	c2	c3	row total
Observed	c1	**2**	1	1	4
	c2	3	**0**	0	3
	c3	0	1	**2**	3
	column total	5	2	3	10

The diagonal sums to $2 + 0 + 2 = 4$, so $L_{0/1} = 1 - 4/10 = 0.6$, the same as the direct count. The matrix also shows that class c2 is never predicted correctly, which the error rate hides. Print the confusion matrix in every report.

5.1 The confusion matrix in code

In R, `table()` names the rows after its *first* argument, so `table(trueVals, preds)` puts the truth on the rows. `diag()` then picks out $2, 0, 2$. In Python, `confusion_matrix` takes the truth first and also puts it on the rows.

R

```r
trueVals <- c("c1","c1","c2","c1","c3",
              "c1","c2","c3","c2","c3")
preds    <- c("c1","c2","c1","c3","c3",
              "c1","c1","c3","c1","c2")
(confMatrix <- table(trueVals, preds))
##         preds
## trueVals c1 c2 c3
##       c1  2  1  1
##       c2  3  0  0
##       c3  0  1  2
(errorRate <- 1 - sum(diag(confMatrix)) /
                  sum(confMatrix))
## [1] 0.6
```

Python

```python
from sklearn.metrics import (confusion_matrix,
                             accuracy_score)
y_true = ["c1","c1","c2","c1","c3",
          "c1","c2","c3","c2","c3"]
y_pred = ["c1","c2","c1","c3","c3",
          "c1","c1","c3","c1","c2"]
labels = ["c1","c2","c3"]
confusion_matrix(y_true, y_pred, labels=labels)
# array([[2, 1, 1],
#        [3, 0, 0],
#        [0, 1, 2]])
1 - accuracy_score(y_true, y_pred)   # 0.6
```

The `caret` package in R gives the same counts through a different call.

```r
library(caret)   # needs e1071 installed
lv <- c("c1", "c2", "c3")
cm <- confusionMatrix(
        factor(preds,    levels = lv),
        factor(trueVals, levels = lv))
cm$table                # PREDICTION on rows
cm$overall["Accuracy"]  # 0.4
```

5.2 Which axis is the truth?

The rule Two of the three common tools put the truth on the rows. One puts the predictions on the rows (Table 5).

`caret::confusionMatrix` takes `data`, the predictions, as its first argument and prints predictions down the rows and the reference across the columns. Its table is the transpose of the one from `table()` and `scikit-learn`, as Table 6 shows.

Table 5. Row and column orientation across three confusion-matrix implementations

Call	Rows	Columns	Matches these notes?
`table(trueVals, preds)`	observed	predicted	yes
`sklearn.metrics.confusion_matrix(y_true, y_pred)`	observed	predicted	yes
`caret::confusionMatrix(data, reference)`	**predicted**	**reference**	**no, transposed**

Table 6. The same confusion matrix with truth on rows, as printed by scikit-learn and R's table function (left), and transposed, as printed by caret (right)

		Predicted		
		c1	c2	c3
Observed	c1	**2**	1	1
	c2	3	**0**	0
	c3	0	1	**2**

		Observed		
		c1	c2	c3
Predicted	c1	**2**	3	0
	c2	1	**0**	1
	c3	1	0	**2**

Why it matters The diagonal is the same in both, so accuracy and the error rate are identical either way and give no warning. Precision and recall (section 8) are not symmetric. Transpose the table by accident and the two swap. Read the printed header (`Prediction` and `Reference`, or `trueVals` and `preds`) before reading any cell.

Differences to know

- **Label order.** `confusion_matrix` sorts the labels alphabetically unless `labels=` is passed, so a `"yes"/"no"` problem comes back with `no` in the first row and column. With character vectors, R's `table()` infers sorted labels from the observed values, so a class absent from both vectors is omitted. Factors keep their declared levels and order. Wrap both vectors in `factor(x, levels = ...)` to keep a fixed matrix shape.
- **Argument order.** `caret::confusionMatrix(data, reference)` takes the predictions first. `confusion_matrix(y_true, y_pred)` takes the truth first. Getting this backwards transposes the table without any error message.

6 Cost-sensitive evaluation

What it is for The error rate assigns unit cost to every mistake and zero cost to every correct prediction. In many applications some mistakes cost far more than others, and some correct predictions are worth more than others.

The definition A **cost/benefit matrix** is a table where each entry specifies the cost (negative benefit) or the benefit of each type of prediction (Table 7). Evaluating a model by its expected benefit follows Provost and Fawcett's expected-value framework.[8]

Table 7. General form of a three-class cost/benefit matrix

		Predicted		
		c_1	c_2	c_3
Observed	c_1	$B_{1,1}$	$C_{1,2}$	$C_{1,3}$
	c_2	$C_{2,1}$	$B_{2,2}$	$C_{2,3}$
	c_3	$C_{3,1}$	$C_{3,2}$	$B_{3,3}$

Benefits B sit on the diagonal and costs C off it. A model is evaluated by the total balance of its predictions, which is the sum of the benefits minus the costs. The cost matrix has the same shape and orientation as the confusion matrix, so the total is the cell-by-cell product, summed.

$$U = \sum_j \sum_k n_{j,k}\, v_{j,k},$$

where $v_{j,k}$ is the cost/benefit entry in row j and column k, a benefit B on the diagonal and a cost C off it.

Worked example In R and in Python, with the running confusion matrix and the cost matrix below.

R

```
costMatrix <- matrix(c(10,-2,-4,-2,30,-3,-5,-6,12),
                     ncol = 3)
colnames(costMatrix) <- c("predC1","predC2",
                          "predC3")
rownames(costMatrix) <- c("obsC1","obsC2","obsC3")
costMatrix
##       predC1 predC2 predC3
## obsC1     10     -2     -5
## obsC2     -2     30     -6
## obsC3     -4     -3     12
(utilityPreds <- sum(confMatrix * costMatrix))
## [1] 28
```

Python

```
import numpy as np
C = np.array([[10, -2, -5],
              [-2, 30, -6],
              [-4, -3, 12]])
M = confusion_matrix(y_true, y_pred,
                     labels=labels)
(M * C).sum()   # 28
```

Cell by cell,

$$\begin{aligned} U &= 2(10) + 1(-2) + 1(-5) + 3(-2) + 0(30) + 0(-6) + 0(-4) + 1(-3) + 2(12) \\ &= 20 - 2 - 5 - 6 - 3 + 24 = 28. \end{aligned}$$

A better error rate, a worse decision Table 8 compares three models on the same test set under the same cost matrix.

Table 8. Error rate and total utility for three models under one cost matrix

model	error rate	total utility
Model A	0.6	+28
always predict c1	0.6	+22
always predict c2	0.7	+73

Two models with an *identical* error rate are worth 28 and 22. Always predicting c1 gives $4(10)+3(-2)+3(-4) = 40-6-12 = 22$. A third model with a *worse* error rate is worth 73, because $B_{2,2} = 30$ makes one correct c2 worth three correct c1s. Always predicting c2 gives $4(-2)+3(30)+3(-3) = -8+90-9 = 73$. The metric defines what winning means, and it is written down before the comparison starts.

Differences to know

- `confMatrix * costMatrix` in R and `M * C` in NumPy are *elementwise* products. `%*%` in R and `@` in NumPy are matrix multiplication and give a meaningless result here.
- R's `matrix()` fills by column unless `byrow = TRUE` is given. The vector `c(10,-2,-4,...)` lays down the first *column* first, which is why the printed matrix does not read in the order the vector was typed. `np.array` takes the rows as written.

- Both matrices must be in the same orientation. Feed `caret`'s transposed table into the product and the running example totals 26 instead of 28.

7 Two classes and a rare event

What it is for Many important problems have two classes, one much less frequent than the other and also the most relevant. Detecting outliers is one example. Fraud, disease and equipment failure are others. With two classes the confusion matrix has four named cells, and several new metrics follow from them.

The definitions Table 9 names the four cells.

Table 9. Definitions of true positive, false negative, false positive and true negative

		Predicted	
		P	N
Observed	P	True Positives (TP)	False Negatives (FN)
	N	False Positives (FP)	True Negatives (TN)

To read a cell name, the *second* word is what the model said and the *first* says whether it was right. A **false negative** is a case the model called negative and was wrong about, a missed event. A **false positive** is a case the model called positive and was wrong about, a false alarm. Rows are still the truth, columns are still the model.

With $N = TP + FP + FN + TN$,

$$\text{Accuracy} = \frac{TP + TN}{N}, \qquad \text{Error rate} = \frac{FP + FN}{N}, \qquad \text{Specificity} = \frac{TN}{TN + FP}, \qquad \text{Prevalence} = \frac{TP + FN}{N}.$$

Worked example Collapse the three classes of the running example to **c1 versus rest**, meaning "is this case a c1?". The c1 row and the c1 column of the 3×3 table give Table 10.

$$TP = 2, \qquad FN = 1 + 1 = 2, \qquad FP = 3 + 0 = 3, \qquad TN = 0 + 0 + 1 + 2 = 3$$

Table 10. The running example collapsed to c1 versus rest

		Predicted	
		c1	not c1
Observed	c1	TP = 2	FN = 2
	not c1	FP = 3	TN = 3

With $N = 10$,

$$\text{Accuracy} = \frac{2 + 3}{10} = 0.5, \qquad \text{Error rate} = \frac{3 + 2}{10} = 0.5,$$

$$\text{Specificity} = \frac{3}{3 + 3} = 0.5, \qquad \text{Prevalence} = \frac{4}{10} = 0.4.$$

The error rate moved It was 0.6 on the three-class table and it is 0.5 here, with the same model. Case 10 (true c3, predicted c2) is an error between two classes that the binary view merges into "not c1", so the binary view counts it as correct. Binarizing hides every confusion inside the negative class. Always state which binarization you used.

8 Precision and recall

What it is for With a rare positive class, accuracy is dominated by the many negatives. Precision and recall look only at the positive class, from the model's side and from the truth's side.

The formulas **Precision** is the proportion of the signals (events) of the model that are correct.

$$Prec = \frac{TP}{TP + FP}$$

Recall is the proportion of the real events that are captured by the model.

$$Rec = \frac{TP}{TP + FN}$$

Precision reads the positive *column* of the truth-on-rows table, everything the model called positive. Recall reads the positive *row*, everything that really was positive (Table 11). This is why transposing the table swaps them.

Table 11. Two-class confusion matrix in symbols. Precision reads the P column, recall reads the P row.

		Predicted	
		P	N
Observed	P	TP	FN
	N	FP	TN

Worked example on a second table The six-case table in Table 12 is a different, smaller data set from the running example. Call it **Example A**. It has $TP = 2$, $FN = 2$, $FP = 1$, $TN = 1$.

Table 12. Confusion matrix for the six-case precision and recall example (Example A)

		Predicted	
		P	N
Observed	P	2	2
	N	1	1

$$Precision = \frac{TP}{TP + FP} = \frac{2}{2+1} = 0.667, \qquad Recall = \frac{TP}{TP + FN} = \frac{2}{2+2} = 0.5,$$

$$ErrorRate = \frac{2+1}{2+2+1+1} = \frac{3}{6} = 0.5.$$

The error-rate numerator is $FN + FP$, the two cells off the diagonal. The denominator is every case.

The same recipe on the running example With c1 as the positive class,

$$Prec = \frac{2}{2+3} = 0.4, \qquad Rec = \frac{2}{2+2} = 0.5, \qquad Spec = \frac{3}{3+3} = 0.5.$$

Table 13 puts the two examples side by side.

Table 13. Precision, recall and related metrics compared across two examples

	Example A (6 cases)	running example (10 cases)
TP / FN / FP / TN	2 / 2 / 1 / 1	2 / 2 / 3 / 3
Precision	0.667	0.400
Recall	0.500	0.500
Specificity	0.500	0.500
Accuracy	0.500	0.500
Error rate	0.500	0.500
F_1	0.571	0.444

Recall, specificity, accuracy and error rate are equal on the two tables by coincidence, which makes the switch of data set easy to miss. Precision and F_1 differ, because precision divides by the predicted-positive count and recall divides by the actual-positive count. From here on, the ten-case running example is used again.

8.1 Precision and recall in code

In R, `caret` reports both once the positive class is named. In Python, each metric has its own function.

R

```r
lv <- c("c1", "rest")
tv <- factor(ifelse(trueVals == "c1", "c1", "rest"),
             levels = lv)
pv <- factor(ifelse(preds    == "c1", "c1", "rest"),
             levels = lv)
# caret prints recall as Sensitivity and
# precision as Pos Pred Value.
confusionMatrix(pv, tv, positive = "c1")
##    Sensitivity : 0.5
##    Specificity : 0.5
## Pos Pred Value : 0.4
## Neg Pred Value : 0.6
```

Python

```python
from sklearn.metrics import (precision_score,
    recall_score, f1_score)
yt = ["c1" if v == "c1" else "rest"
      for v in y_true]
yp = ["c1" if v == "c1" else "rest"
      for v in y_pred]
precision_score(yt, yp, pos_label="c1")  # 0.4
recall_score(yt, yp, pos_label="c1")     # 0.5
f1_score(yt, yp, pos_label="c1")         # 0.444
```

Both calls return the same numbers only because the positive class is named. Two defaults decide what you get when it is not, and the two tools name the metrics differently.

Differences to know

- **Which class is positive?** `caret::confusionMatrix` takes the *first level of the factor* as positive unless `positive=` is supplied, and factor levels are alphabetical by default. In a `"clean"` versus `"fraud"` problem, `clean` silently becomes the positive class and the reported recall is the recall of the wrong class. `scikit-learn` defaults to `pos_label=1` and raises an error on string labels rather than guessing. Set the positive class explicitly in both.
- **Zero denominator.** If a class is never predicted, $TP+FP = 0$ and precision is $0/0$. `scikit-learn` returns 0.0 and emits an `UndefinedMetricWarning` (the default is `zero_division="warn"`). Pass `zero_division=0`, `1` or `np.nan` to choose the behaviour. `caret` returns `NaN` for *Pos Pred*

Value (and `NA` for *Precision* under `mode = "prec_recall"`). The same model on the same data scores `0` in one language and `NaN` in the other. In the running example c2 is predicted twice, so this does not arise, but removing both c2 predictions would make its precision undefined.

- **Vocabulary.** `caret` prints *Sensitivity* for recall and *Pos Pred Value* for precision. These are the same numbers under the names used in epidemiology.

9 The F-score

What it is for There is a trade-off between precision and recall. It is easy to improve recall if we ignore precision, and vice versa. A model that predicts "positive" for every case has perfect recall and precision equal to the positive-class prevalence. A model that predicts positive only for the one case it is surest about usually has the reverse. Selecting a model on two metrics is hard, so the F-score combines them into one.

The formula

$$F_\beta = \frac{(\beta^2 + 1) \cdot Prec \cdot Rec}{\beta^2 \cdot Prec + Rec}$$

F_1 is the harmonic mean of precision and recall, and F_β is a weighted harmonic mean, the complement $1 - E$ of van Rijsbergen's effectiveness measure E.[9] The user chooses β, with default 1. When $\beta \to 0$ the weight of recall decreases. When $\beta \to \infty$ the weight of precision decreases.

Worked example At $\beta = 1$ the formula becomes $2PR/(P + R)$. On the running example, $P = 0.4$ and $R = 0.5$, so

$$F_1 = \frac{2(0.4)(0.5)}{0.4 + 0.5} = \frac{0.4}{0.9} = 0.444.$$

On Example A, $P = 0.667$ and $R = 0.5$, so

$$F_1 = \frac{2(0.667)(0.5)}{0.667 + 0.5} = \frac{0.667}{1.167} = 0.571.$$

The `f1_score` call in the previous section returns the first value.

Why a harmonic mean The harmonic mean sits close to the *smaller* of its arguments. A model with precision 1.0 and recall 0.01 has an arithmetic mean of 0.505 but an F_1 of only $2(1)(0.01)/(1.01) \approx 0.0198$. The models that maximize one metric by giving up the other are exactly the ones F_1 is designed to score low. Figure 5 draws curves of constant F_1. Moving along a curve trades one metric for the other with no change in F_1.

Choosing β The relative weight of recall in the harmonic mean is β^2. In counts,

$$F_\beta = \frac{(1 + \beta^2)TP}{(1 + \beta^2)TP + \beta^2 FN + FP}.$$

For $\beta = 2$ the denominator weights FN by 4 and FP by 1. This favours recall, but it is not a monetary cost ratio. When the actual costs are known, use a cost/benefit matrix. Table 14 sweeps β for both

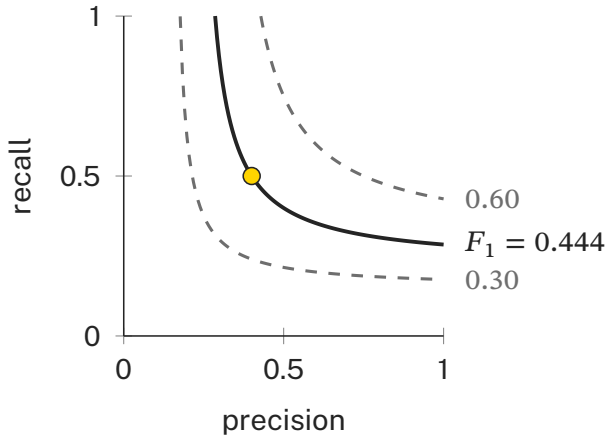


Figure 5. Curves of constant F_1 in the precision and recall plane. The marked point is the running example, $F_1 = 0.444$.

Table 14. Values of F_β at three weightings for both examples

	running example $P = 0.400$, $R = 0.500$	Example A $P = 0.667$, $R = 0.500$
$F_{0.5}$ (precision weighted more)	0.417	0.625
F_1 (balanced)	0.444	0.571
F_2 (recall weighted more)	0.476	0.526

examples. In the running example precision is the lower value, so raising β raises the score. In Example A recall is the lower value, so raising β lowers it.

For example, on the running example, $F_2 = 5(0.4)(0.5)/(4(0.4) + 0.5) = 1/2.1 = 0.476$. Figure 6 traces F_β for Example A as β grows. It starts at precision and falls towards recall.

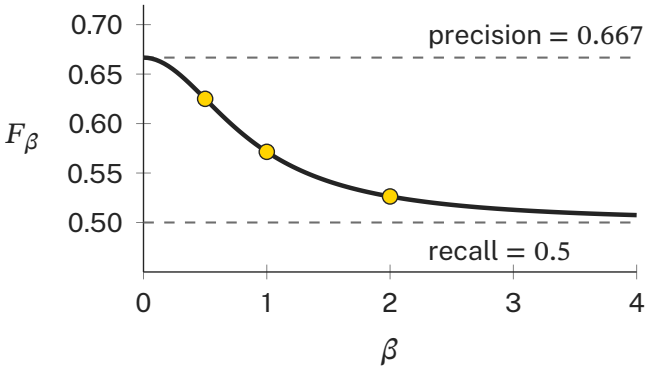


Figure 6. F_β against β for Example A, with $Prec = 0.667$ and $Rec = 0.5$. The marked points are $\beta = 0.5$, 1 and 2.

10 More than two classes: macro, micro and weighted averaging

What it is for Precision and recall are defined one class at a time. A three-class problem has three of each, so reporting a single precision requires an averaging rule. Sokolova and Lapalme analyse the macro and micro forms systematically.[11]

The three rules

- **Macro** averages the per-class values with equal weight. Use it when a rare class matters as much as a common one.
- **Micro** pools the counts of all classes first and then divides. For single-label multi-class problems micro precision, micro recall and micro F_1 are all equal to accuracy.
- **Weighted** averages the per-class values by support, the number of true cases in each class. Weighted recall is always equal to accuracy.

Worked example Treat each class of the running example in turn as positive (Table 15).

Table 15. Per-class and averaged precision, recall and F_1 for three classes

class	TP	FP	FN	support	Precision	Recall	F_1
c1	2	3	2	4	0.400	0.500	0.444
c2	0	2	3	3	0.000	0.000	0.000
c3	2	1	1	3	0.667	0.667	0.667
macro average					0.356	0.389	0.370
micro average	4	6	6	10	0.400	0.400	0.400
weighted average					0.360	0.400	0.378

Macro precision is $(0.400 + 0 + 0.667)/3 = 0.356$, so class c2's zero counts as much as class c3's 0.667. Micro precision pools the counts, $4/(4 + 6) = 0.4$, which is the accuracy. Weighted precision uses the supports $(4, 3, 3)$, giving $(4(0.400) + 3(0) + 3(0.667))/10 = 0.360$.

In code `scikit-learn` prints all three in one report.

```
from sklearn.metrics import classification_report
print(classification_report(y_true, y_pred,
      labels=labels, digits=3, zero_division=0))
#               precision  recall  f1-score  support
#           c1      0.400   0.500     0.444        4
#           c2      0.000   0.000     0.000        3
#           c3      0.667   0.667     0.667        3
#     accuracy                        0.400       10
#    macro avg      0.356   0.389     0.370       10
# weighted avg      0.360   0.400     0.378       10
```

Differences to know

`caret` has no single call that prints this block. `caret::confusionMatrix(...)$byClass` returns the per-class rows but *no* averages. You take the column means yourself, and so you must choose macro or weighted explicitly.

11 One threshold, or all of them

What it is for Every metric above scored one fixed set of predictions. Most classifiers emit a score, and a threshold turns the score into a class. Move the threshold and the whole confusion matrix moves with it, and every metric above changes. The ROC curve and the area under it (AUC) score the ranking across every threshold at once.[2]

The definition For each threshold, plot the true positive rate (recall) against the false positive rate (1− specificity). The ROC curve joins these points. The AUC is the area under that curve. A model that ranks at random lies on the diagonal and has AUC 0.5.

Worked example Suppose the model that produced the running example also reported $\hat{P}(c_1)$, as in Table 16.

Table 16. Predicted probability of class c1 for the ten-case running example

case	1	2	3	4	5	6	7	8	9	10
truly c1?	yes	yes	no	yes	no	yes	no	no	no	no
$\hat{P}(c_1)$	0.91	0.47	0.74	0.31	0.28	0.62	0.58	0.12	0.53	0.09

Cutting at 0.5 predicts c1 for cases 1, 3, 6, 7 and 9. That reproduces the c1 column of the running example, $TP = 2$, $FP = 3$, so recall $= 0.5$ and $1 - \text{specificity} = 0.5$. This single operating point sits on the chance diagonal. Sweeping the threshold over all ten scores traces the ROC curve through $(0, 0.25)$, $(1/6, 0.25)$, $(1/6, 0.5)$, $(0.5, 0.5)$, $(0.5, 1)$ and $(1, 1)$ (Figure 7). The area under it is $17/24 = 0.708$. The AUC equals the share of (c1, not c1) pairs in which the c1 case has the higher score,[3] and 17 of the $4 \times 6 = 24$ pairs are ordered correctly. The ranking the model produces is better than its accuracy at this one threshold suggests, so report both.

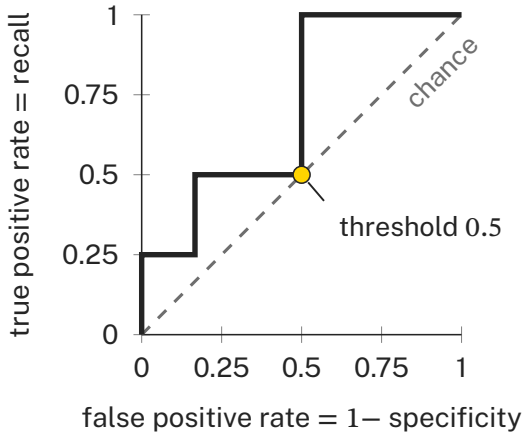


Figure 7. ROC curve for the running example. The marked point is the threshold 0.5, on the chance diagonal. AUC $= 17/24 = 0.708$.

In code In R, `pROC` builds the curve and its area. In Python, `roc_curve` returns the points and `roc_auc_score` the area.

R

```r
library(pROC)
scores <- c(0.91, 0.47, 0.74, 0.31, 0.28,
            0.62, 0.58, 0.12, 0.53, 0.09)
r <- roc(response  = ifelse(trueVals == "c1", 1, 0),
         predictor = scores, levels = c(0, 1),
         direction = "<")
auc(r)   # 0.7083
```

Python

```python
from sklearn.metrics import roc_curve, roc_auc_score
scores = [0.91, 0.47, 0.74, 0.31, 0.28,
          0.62, 0.58, 0.12, 0.53, 0.09]
y = [1 if v == "c1" else 0 for v in y_true]
fpr, tpr, thr = roc_curve(y, scores)
roc_auc_score(y, scores)   # 0.7083
```

Differences to know

`pROC::roc()` chooses `direction` automatically by comparing the control and case medians, and prints the choice. This does not guarantee AUC ≥ 0.5. `roc_auc_score` uses the supplied score direction. Feed it a reversed score and it returns $0.292 = 7/24$, correctly reporting that the ranking is inverted. Set `levels=` and `direction=` explicitly in `pROC` and the two agree.

12 Common misconceptions

1. **"The confusion matrix has truth on the rows, always."** It has truth on the rows in `table(truth, prediction)` and in `sklearn`. It has *predictions* on the rows in `caret::confusionMatrix`. Read the header.
2. **"Accuracy is a fine default metric."** On the running example accuracy is 0.4 and hides that class c2 is never predicted correctly. On an imbalanced problem (1% fraud) a model that predicts "clean" for everything scores 99%.
3. **"A lower error rate means a better model."** Only if all errors cost the same. Under the cost matrix of section 6 the always-c2 model has a worse error rate (0.7 versus 0.6) and more than twice the utility (73 versus 28).
4. **"Precision and recall are two views of the same thing."** They read different margins of the table. Precision has FP in its denominator and recall has FN, so a model can move one without touching the other.
5. **"F_1 is the average of precision and recall."** It is the *harmonic* mean, which is closer to the smaller value.
6. **"Binarizing a three-class problem does not change anything."** It changed the error rate here from 0.6 to 0.5, because it merged two classes and ignored the confusion between them.
7. **"The metric can be picked once the results are in."** Then the conclusion was chosen first. Step 1 of model selection comes before step 2.
8. **"A class with no predictions just gets a precision of zero."** It gets $0/0$. What is reported depends on the library and on the `zero_division` setting.

13 Summary

- Prediction models approximate the relationship between explanatory variables and a target variable.
- A categorical target makes a classification problem. A numeric target makes a regression problem.
- Evaluation metrics are used to help select a model, and they are chosen before the comparison. The most appropriate metric depends on the application, its costs, its class balance and what an error means.
- For classification, the confusion matrix is the basic object, and every metric up to the ROC curve is a ratio read from it. The error rate and accuracy read the diagonal. Precision reads a column, recall reads a row, specificity reads the other row, and F_β combines precision and recall with a stated weight.
- Cost/benefit matrices drop the assumption that all errors are equal, and then a lower error rate can be the worse decision.

- With more than two classes, the averaging rule (macro, micro or weighted) must be chosen and named.
- The orientation of the printed table decides whether your precision is precision or recall.

References

[1] P. Chapman, J. Clinton, R. Kerber, T. Khabaza, T. Reinartz, C. Shearer, and R. Wirth. *CRISP-DM 1.0: Step-by-step Data Mining Guide*. Tech. rep. The CRISP-DM Consortium, 2000.

[2] T. Fawcett. "An introduction to ROC analysis". In: *Pattern Recognition Letters* 27.8 (2006), pp. 861–874.

[3] J. A. Hanley and B. J. McNeil. "The meaning and use of the area under a receiver operating characteristic (ROC) curve". In: *Radiology* 143.1 (1982), pp. 29–36.

[4] G. James, D. Witten, T. Hastie, and R. Tibshirani. *An Introduction to Statistical Learning*. 2nd ed. Springer, 2021.

[5] M. Kuhn. *caret: Classification and Regression Training*. Version 7.0-1. Package documentation for model training, resampling, tuning and evaluation. CRAN. 2024.

[6] D. Meyer, E. Dimitriadou, K. Hornik, A. Weingessel, and F. Leisch. *e1071: Misc Functions of the Department of Statistics, Probability Theory Group (Formerly: E1071), TU Wien*. Version 1.7-17. Documentation for `svm` and `predict.svm`. CRAN. 2025.

[7] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay. "Scikit-learn: machine learning in Python". In: *Journal of Machine Learning Research* 12.85 (2011), pp. 2825–2830.

[8] F. Provost and T. Fawcett. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly, 2013.

[9] C. J. van Rijsbergen. *Information Retrieval*. 2nd ed. London: Butterworths, 1979.

[10] X. Robin, N. Turck, A. Hainard, N. Tiberti, F. Lisacek, J.-C. Sanchez, and M. Müller. "pROC: an open-source package for R and S+ to analyze and compare ROC curves". In: *BMC Bioinformatics* 12 (2011), p. 77.

[11] M. Sokolova and G. Lapalme. "A systematic analysis of performance measures for classification tasks". In: *Information Processing & Management* 45.4 (2009), pp. 427–437.

[12] L. Torgo. *Data Mining with R: Learning with Case Studies*. 2nd ed. Chapman and Hall/CRC, 2016.